

Contrôle des connaissances de programmation C++

L1 informatique – durée 1h – Aucun document n'est autorisé

CORRECTION

Rappel : Soient `s1` et `s2` des chaînes de caractères de type `string`.

`s1 <= s2` retourne `true` si `s1` est alphabétiquement inférieure ou égale à `s2`, sinon `false`.

`s1 == s2` retourne `true` si `s1` et `s2` ont le même contenu alphabétique, sinon `false`.

Structures données :

```
1 class STRING {
2     string mot;
3     STRING * suivant;
4 };
5
6 class LISTE_STRINGS {
7     STRING * tete;
8     STRING * queue;
9 };
10
11 class MOT {
12     string mot;
13     LISTE_STRINGS LS;
14 };
```

Question 1. Constructeur de STRING

Correction

```
1 STRING::STRING(const string & m) {
2     mot = m;
3     suivant = nullptr;
4 }
```

On initialise `mot` avec la chaîne passée en argument et `suivant` à `nullptr` (ce maillon ne pointe vers rien initialement).

Question 2. Constructeur sans argument de LISTE_STRINGS

Correction

```
1 LISTE_STRINGS::LISTE_STRINGS() {
2     tete = nullptr;
3     queue = nullptr;
4 }
```

Une liste vide est représentée par `tete` et `queue` tous deux à `nullptr`.

Question 3. LISTE_STRINGS::ajout_queue(string s)

Correction

```
1 void LISTE_STRINGS::ajout_queue(const string & s) {  
2     STRING * nouveau = new STRING(s);  
3     if (tete == nullptr) {  
4         // Liste vide : le nouveau maillon est a la fois tete et  
5         queue  
6         tete = nouveau;  
7         queue = nouveau;  
8     } else {  
9         // On chaine apres la queue actuelle  
10        queue->suivant = nouveau;  
11        queue = nouveau;  
12    }
```

Cas particulier : si la liste est vide, tete et queue doivent toutes deux pointer vers le nouveau maillon.

Question 4. Constructeur de MOT

Correction

Le constructeur prend en argument une chaîne de caractères, un tableau de chaînes et la taille du tableau.

```
1 MOT::MOT(const string & m, string tab[], int taille) {  
2     mot = m;  
3     // LS est initialise automatiquement par son propre  
4     constructeur  
5     // (tete = nullptr, queue = nullptr)  
6     for (int i = 0; i < taille; i++) {  
7         LS.ajout_queue(tab[i]);  
8     }
```

L'attribut LS de type LISTE_STRINGS est un **attribut automatique** : son constructeur sans argument est appelé automatiquement. On remplit ensuite la liste avec les synonymes du tableau.

Question 5. Destruction d'un objet MOT – fonctions nécessaires

Correction

Démarche : La classe MOT contient un attribut LS de type LISTE_STRINGS, qui lui-même contient des pointeurs vers des maillons STRING alloués dynamiquement avec `new`. Le destructeur par défaut ne libère pas ces allocations il faut écrire des destructeurs explicites.

1. Destructeur de LISTE_STRINGS : parcours de la liste et `delete` de chaque maillon.
2. Destructeur de MOT : appel automatique du destructeur de LS (attribut automatique).

```

1 // Destructeur de LISTE_STRINGS
2 LISTE_STRINGS::~~LISTE_STRINGS() {
3     STRING * courant = tete;
4     while (courant != nullptr) {
5         STRING * suivant = courant->suivant;
6         delete courant;
7         courant = suivant;
8     }
9     tete = nullptr;
10    queue = nullptr;
11 }
12
13 // Destructeur de STRING (optionnel mais propre)
14 STRING::~~STRING() {
15     suivant = nullptr;
16 }
17
18 // Destructeur de MOT
19 // Le destructeur de LS est appele automatiquement
20 MOT::~~MOT() {
21     // Rien de particulier : ~LISTE_STRINGS() est appele
22     // automatiquement
23 }
```

Question 6. Constructeur par recopie de MOT

Correction

Le constructeur par recopie doit effectuer une **copie profonde** de la liste chaînée (et non une simple copie de pointeurs).

```

1 MOT::MOT(const MOT & m) {
2     mot = m.mot;
3     // Parcours de la liste de m pour recopier chaque maillon
4     STRING * p = m.LS.tete;
5     while (p != nullptr) {
6         LS.ajout_queue(p->mot);
7         p = p->suivant;
8     }
9 }
```

Attention : copier uniquement les pointeurs sinon double `delete` à la destruction.

Question 7. Opérateur d'affectation de MOT

Correction

```

1 MOT & MOT::operator=(const MOT & m) {
2     if (this == &m) return *this; // auto-affectation
3     // Liberation de la liste existante
4     mot = m.mot;
5     // On reecrit LS : on detruit l'ancienne et on recopie
6     LISTE_STRINGS nouvelle; // liste vide
7     STRING * p = m.LS.tete;
8     while (p != nullptr) {
9         nouvelle.ajout_queue(p->mot);
10        p = p->suivant;
11    }
12    LS = nouvelle; // appelle l'operateur d'affectation de
13    LISTE_STRINGS
14    return *this;
15 }
16 // Il faut aussi l'operateur d'affectation de LISTE_STRINGS :
17 LISTE_STRINGS & LISTE_STRINGS::operator=(const LISTE_STRINGS & L) {
18     if (this == &L) return *this;
19     // Destruction de l'ancienne liste
20     STRING * courant = tete;
21     while (courant != nullptr) {
22         STRING * suiv = courant->suivant;
23         delete courant;
24         courant = suiv;
25     }
26     tete = queue = nullptr;
27     // Recopie profonde
28     STRING * p = L.tete;
29     while (p != nullptr) {
30         ajout_queue(p->mot);
31         p = p->suivant;
32     }
33     return *this;
34 }

```

Question 8. LISTE_STRINGS::In_liste(string s)

Correction

```

1 bool LISTE_STRINGS::In_liste(const string & s) const {
2     STRING * p = tete;
3     while (p != nullptr) {
4         if (p->mot == s) return true;
5         p = p->suivant;
6     }
7     return false;
8 }

```

On parcourt la liste maillon par maillon et on compare chaque mot à s.

Question 9. Fonction Est_synonyme

Correction

```

1 bool Est_synonyme(const MOT & M1, const MOT & M2) {
2     // M1 est synonyme de M2 si :
3     // - le mot de M1 appartient aux synonymes de M2, OU
4     // - le mot de M2 appartient aux synonymes de M1
5     return M2.LS.In_liste(M1.mot) || M1.LS.In_liste(M2.mot);
6 }

```

La relation est symétrique par définition de l'énoncé.

Question 10. LISTE_STRINGS::insere_liste_triee(string s)

Correction

```

1 void LISTE_STRINGS::insere_liste_triee(const string & s) {
2     STRING * nouveau = new STRING(s);
3
4     // Cas 1 : liste vide
5     if (tete == nullptr) {
6         tete = nouveau;
7         queue = nouveau;
8         return;
9     }
10
11     STRING * courant = tete;
12     STRING * precedent = nullptr;
13
14     // Avancer tant que s > courant->mot
15     while (courant != nullptr && courant->mot <= s) {
16         precedent = courant;
17         courant = courant->suivant;
18     }
19
20     // Cas 2 : insertion en tete (precedent == nullptr)
21     if (precedent == nullptr) {
22         nouveau->suivant = tete;
23         tete = nouveau;
24     }
25     // Cas 3 : insertion en queue (courant == nullptr)
26     else if (courant == nullptr) {
27         precedent->suivant = nouveau;
28         queue = nouveau;
29     }
30     // Cas 4 : insertion au milieu
31     else {
32         precedent->suivant = nouveau;
33         nouveau->suivant = courant;
34     }
35 }

```

Rappel des cas à ne pas oublier :

— Liste vide → tete et queue pointent sur le nouveau maillon.

- Insertion avant le premier élément → mise à jour de `tete`.
- Insertion après le dernier élément → mise à jour de `queue`.

Question 11. Tri alphabétique de la liste (LISTE_STRINGS)

Correction

```
1 void LISTE_STRINGS::trier() {
2     LISTE_STRINGS nouvelle; // liste vide, triee
3
4     STRING * courant = tete;
5     while (courant != nullptr) {
6         nouvelle.insere_liste_triee(courant->mot);
7         courant = courant->suivant;
8     }
9
10    // On remplace la liste actuelle par la liste triee
11    *this = nouvelle; // utilise l'operateur d'affectation
12 }
```

On crée une nouvelle liste vide, on y insère chaque élément de la liste originale via `insere_liste_triee` (qui maintient l'ordre), puis on affecte le résultat à `*this`.

Question 12. Tri des synonymes d'un MOT

Correction

```
1 void MOT::trier_synonymes() {
2     LS.trier();
3 }
```

Il suffit de déléguer le tri à la méthode `trier()` de l'attribut `LS`.